Université Cadi Ayyad
Faculté des Sciences et Techniques de Marrakech

MST IA2I

Compte Rendu des TP : Kotlin

Réalisé par :

ARJAZ DRAOUI Douae

Année Universitaire
2025/2026

Contents

List of Figures **2**

1 TP1 : Gestion de Facture **2**
1.1 Objectif 2
1.2 Classes et Implémentation 2
1.2.1 Classe Produit 2
1.3 Fonction main et Menu 2

2 TP2 : Gestion d'un Parking **3**
2.1 Objectif 3
2.2 Classes et Implémentation 3
2.2.1 Classe Utilisateur 3
2.2.2 Classe Véhicule et ses héritiers 3
2.2.3 Classe Parking (abstraite) et ParkingTab 4
2.3 Fonction main et Menu 4

1 TP1 : Gestion de Facture

1.1 Objectif

L'objectif du TP était de concevoir un programme en Kotlin permettant de gérer des factures, chaque facture contenant un ensemble de produits. Chaque produit est caractérisé par :

- une référence,
- une désignation,
- une quantité,
- un prix unitaire,
- le total produit.

1.2 Classes et Implémentation

.

1.2.1 Classe Produit

La classe `Produit` a été conçue avec un constructeur principal et des **setters** personnalisés pour valider les valeurs et formater les chaînes :

```
class Produit(
    reference: String,
    designation: String,
    quantite: Int,
    prixUnitaire: Double
) {
    var reference: String = reference
        set(value) { field = value.uppercase() }

    var designation: String = designation
        set(value) { field = value.replaceFirstChar { it.uppercase() } }

    var quantite: Int = quantite
        set(value) { if(value > 0) field = value else println("Quantit  invalide
            ") }

    var prixUnitaire: Double = prixUnitaire
        set(value) { if(value > 0) field = value else println("Prix invalide") }

    val totalProduit: Double
        get() = quantite * prixUnitaire

    // Constructeur secondaire pour les services
    constructor(reference: String, designation: String, prixUnitaire: Double)
        : this(reference, designation, 1, prixUnitaire)
}
```

1.3 Fonction main et Menu

Un tableau `Array<Produit?>` est utilisé pour stocker les produits. Un menu interactif permet d'ajouter des produits ou des services, et d'afficher la facture :

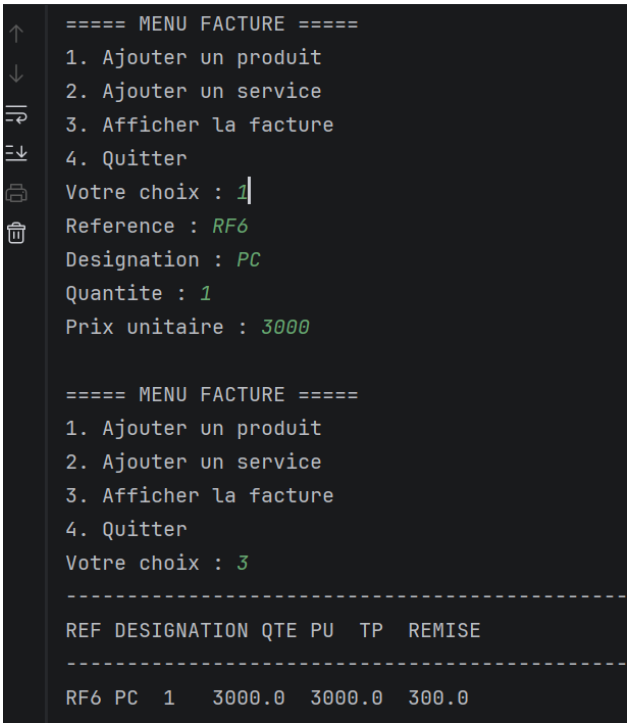

Figure 1: Affichage du resultat

2 TP2 : Gestion d'un Parking

2.1 Objectif

Le TP visait à modéliser un parking avec différents types de véhicules (voitures et camions) et des utilisateurs. Chaque véhicule possède une date d'arrivée et de sortie. Le parking devait être géré dans un tableau avec prise en compte de l'occupation (un camion occupe 2 places).

2.2 Classes et Implémentation

2.2.1 Classe Utilisateur

```
data class Utilisateur(val nom: String, val motDePasse: String)
```

2.2.2 Classe Véhicule et ses héritiers

```
open class Vehicule(
    val conducteur: Utilisateur,
    val marque: String,
    val modele: String,
    var dateArrivee: String,
    var dateSortie: String? = null
) {
    open fun afficheInfos(): String {
        return "Conducteur: ${conducteur.nom}, Marque: $marque, Mod le: $modele,
            " +
                "Arriv e: $dateArrivee, Sortie: ${dateSortie ?: "en cours"}"
    }
}
```

```
class Voiture(...) : Vehicule(...) { ... }
class Camion(...) : Vehicule(...) { ... }
```

2.2.3 Classe Parking (abstraite) et ParkingTab

```
abstract class Parking {
    abstract fun ajouteVehicule(v: Vehicule)
    abstract fun retireVehicule(index: Int)
    abstract fun afficheInfos()
}

class ParkingTab(val capacite: Int) : Parking() {
    private val tabVehicules = arrayOfNulls<Vehicule>(capacite)
    private var compteur = 0

    override fun ajouteVehicule(v: Vehicule) { ... }
    override fun retireVehicule(index: Int) { ... }
    override fun afficheInfos() { ... }
}
```

2.3 Fonction main et Menu

Un menu interactif permet :

- l'ajout de voitures et camions,
- le retrait d'un véhicule par index,
- l'affichage des informations de tous les véhicules,
- la gestion de la capacité du parking.

```
fun main() {
    val parking = ParkingTab(10)
    val user1 = Utilisateur("Alice", "pwd1")
    val user2 = Utilisateur("Bob", "pwd2")

    do {
        println("1. Ajouter une voiture")
        println("2. Ajouter un camion")
        println("3. Retirer un v hicule")
        println("4. Afficher le parking")
        println("5. Quitter")
        val choix = readln().toInt()
        when(choix) { ... }
    } while (choix != 5)
}
```

```
===== MENU PARKING =====
1. Ajouter une voiture
2. Ajouter un camion
3. Retirer un véhicule
4. Afficher le parking
5. Quitter
Votre choix : 1
Marque : volksvagen
Modèle : 2010
Nombre de places : 5
Véhicule ajouté : Conducteur: Alice, Marque: volksvagen, Modèle: 2010, Arrivée: 12/02/2026, S
```

Figure 2: Resultat 1

```
===== MENU PARKING =====
1. Ajouter une voiture
2. Ajouter un camion
3. Retirer un véhicule
4. Afficher le parking
5. Quitter
Votre choix : 4
=== Parking ===
Place 0 : Conducteur: Alice, Marque: volksvagen, Modèle: 2010, Arrivée: 12/02/2026, Sortie: e
Place 1 : Conducteur: Bob, Marque: mitsubichi, Modèle: 2010, Arrivée: 12/02/2026, Sortie: en
Nombre de véhicules: 2
```

Figure 3: Resultat 2